Phase 3 — Protection Analysis: What Safeguards Exist?

Objective: Evaluate every security and reliability control present in the MCP stack, from the stdio transport layer through to the PostgreSQL backend.

Rating Key

Rating	Meaning
Strong	Defense-in-depth, properly designed, minimal residual risk
Adequate	Functional, covers the primary threat, limited gaps
Weak	Present but easily bypassed, incomplete, or covers a narrow case
Absent	No control exists

---

1. Authentication Layer

1.1 Bearer Token Mechanism

Rating: Adequate

Evidence (`main.go:32–38`):

```
if *token == "" {
    *token = os.Getenv("ICHOR_TOKEN")
}
if *token == "" {
    fmt.Fprintln(os.Stderr, "error: --token flag or ICHOR_TOKEN environment variable required")
    os.Exit(1)
}
```

What works well:

- Hard fail on startup if no token is provided. The server won't run without authentication. This prevents accidental anonymous access.
- Two injection paths: CLI flag (`--token`) and environment variable (`ICHOR_TOKEN`). The env var path is the safer option.
- Token is stored in-memory in `Client.token` (`client/ichor.go:17`) and never written to disk by the MCP server itself.
- Every HTTP request injects the token as `Authorization: Bearer {token}` (`client/ichor.go:41`). No request reaches Ichor without the credential.

What is weak or absent:

- **CLI flag in process list**: When `--token` is used, the token value appears in `ps aux` / `/proc/{pid}/cmdline` and is visible to any user on the system with process-list access. The README demonstrates exactly this risk pattern: `go run ./cmd/ichor-mcp/ --token $ICHOR_TOKEN`.
- **Claude Desktop config (`README.md:44–55`)**: The documented Claude Desktop configuration passes the token via `"args": ["--token", "YOUR_TOKEN"]`. This stores the token in plaintext in `~/Library/Application Support/Claude/claude_desktop_config.json`. Default macOS user-level permissions (mode 644 or similar) mean any process running as the same user can read it.
- **No token format validation**: The MCP server does not verify the token is a valid JWT before starting. A garbage token starts the server and fails on the first API call, surfacing the error to the agent as a tool failure.

1.2 Token Lifecycle

Rating: Weak

What works well:

- JWT expiry is enforced by Ichor's auth service. An expired token returns HTTP 401, which the MCP client propagates as a tool error.

What is absent:

- **No expiry handling**: The MCP server has no token refresh logic. When a token expires mid-session, all subsequent tool calls fail with HTTP 401. There is no graceful recovery — the server process must be restarted with a new token.
- **No rotation capability**: Once started, the server runs with a single static token. Token rotation (e.g., exchanging a short-lived token for a new one) is not possible without restarting the process.
- **No revocation detection**: If an admin revokes the token (e.g., due to suspected compromise), the server continues attempting to use it until every call returns 401. There is no shutdown hook or alert on 401 responses.

---

2. Authorization Layer

2.1 API-Delegated RBAC

Rating: Adequate

Evidence (`introspectionapi/routes.go:34–65`): All REST endpoints use `mid.Authenticate()` + `mid.Authorize()` middleware with explicit rule constants:

- Schema introspection endpoints: `auth.RuleAdminOnly`
- Enum options (`/v1/config/enums/{schema}/{name}/options`): `auth.RuleAny`

What works well:

- Authorization is enforced consistently by the REST API regardless of how the request arrives (MCP, curl, frontend). The MCP server cannot bypass it.
- Admin-only introspection is meaningful: a non-admin agent cannot call `search_database_schema` successfully.
- The MCP server has no authorization bypass paths — it cannot forge credentials, escalate privileges, or change the token it uses at runtime.

What is absent:

- **No per-tool authorization at MCP level**: The MCP server registers all tools for a context mode without checking the caller's role. A token with limited permissions still receives all 33 tool definitions from the server; it just fails when calling tools its role doesn't permit. This is a least-privilege violation — tools should ideally not be offered if the caller cannot execute them.

2.2 Authorization Matrix for Key Tools

Tool	REST Endpoint	Auth Rule	Effective Gate
`search_database_schema`	`/v1/introspection/*`	`RuleAdminOnly`	Admin tokens only
`search_enums` (list types)	`/v1/introspection/enums/{schema}`	`RuleAdminOnly`	Admin tokens only
`search_enums` (get values)	`/v1/config/enums/{schema}/{name}/options`	`RuleAny`	Any authenticated token

Tool	REST Endpoint	Auth Rule	Effective Gate
`create_workflow`	`POST /v1/workflow/rules/full`	Table permission (write)	Token must have workflow write access
`validate_workflow`	`POST /v1/workflow/rules/full?dry_run=true`	Table permission (write)	Write permission likely required even for dry-run
`get_workflow`	`GET /v1/workflow/rules/{id}`	Table permission (read)	Token must have workflow read access
All UI write tools	`/v1/config/*`, `/v1/data/*`	Table permission (write)	Token must have config write access

Notable discrepancy: `search_enums` in `search.go:85` calls `c.GetEnumOptions()` which maps to `/v1/config/enums/{schema}/{name}/options` — gated by `auth.RuleAny`. But `c.GetEnumTypes()` (`search.go:78`) calls `/v1/introspection/enums/{schema}` — gated by `auth.RuleAdminOnly`. The `search_enums` tool therefore has an **asymmetric auth level** depending on whether the caller provides a `name` argument or not. Listing enum types in a schema requires admin; reading specific enum values does not.

---

3. Input Validation Controls

3.1 Required Field Checks

Rating: Adequate

Evidence (`read_workflow.go:22` , `write_workflow.go:105–107`):

```
if args.ID == "" {
    return errorResult("id is required"), nil, nil
}
```

What works well:

- Every tool that takes an ID parameter has an explicit empty-string guard before making any HTTP call.
- The MCP Go SDK auto-parses `Required: []string{"workflow"}` from the tool's `InputSchema`, providing a first layer of validation for structured inputs.

What is absent:

- **No UUID format validation**: `args.ID` is accepted as any non-empty string and spliced directly into URL paths (`"/v1/workflow/rules/"+id`). An agent (or a prompt-injected agent) could pass `../admin` as an ID. However, this is mitigated by Go's `net/http` router which treats path components opaquely — so `../../etc/passwd` won't traverse the filesystem. The API backend would return 404 or 400.
- **No maximum payload size**: `json.RawMessage` payloads for `workflow`, `config`, `form`, etc. are forwarded to the REST API with no size limit checked at the MCP level. The 30-second timeout provides an indirect bound (large payloads time out), but there is no explicit cap.

- **No JSON schema validation of write payloads**: The workflow JSON object passed to `create_workflow` is forwarded to the API as-is. The MCP layer does not validate that it matches the expected workflow schema before sending. Validation happens at the backend via `dry_run=true`, but only if `validate=false` is not set.

3.2 Validate-First Pattern

Rating: Adequate for workflows, Absent for UI writes

Evidence (`write_workflow.go:56–77`):

```
shouldValidate := args.Validate == nil || *args.Validate
if shouldValidate {
    valResult, err := c.ValidateWorkflow(ctx, args.Workflow)
    ...
    if !result.Valid {
        return jsonResult(validationErrorResponse), nil, nil
    }
}
```

What works well:

- `create_workflow` and `update_workflow` default to `validate=true` (`write_workflow.go:56` : `args.Validate == nil || *args.Validate`). An agent must explicitly opt out.
- The `validate_workflow` tool exists as a dedicated dry-run step.
- `validate_table_config` provides a similar gate for table configurations.

What is weak:

- **`validate=false` bypass is fully exposed**: The tool description for `create_workflow` explicitly documents: **"Set validate=false to skip pre-validation."** This is the developer escape hatch made a first-class user-facing feature. Any agent — including prompt-injected ones — can pass `{"validate": false}` to create an unvalidated workflow. There is no admin-only gate on this parameter.
- **Validation catches structural problems, not semantic ones**: A workflow that is a valid DAG with correct action configurations but semantically harmful (e.g., `send_email` to a large recipient list on every order_update) will pass validation and be created.

What is absent:

- **No validate-first for UI writes**: `create_page_config`, `create_form`, `add_form_field`, `create_table_config`, `update_page_config`, `create_page_content` etc. — none of these have a validation gate equivalent to workflow's dry-run pattern. They write directly on the first attempt.

3.3 Resource URI Parsing

Rating: Adequate

Evidence (`resources.go:150–167`):

```
func parseDBResourceURI(uri string) (string, string, error) {
    const prefix = "config://db/"
    ...
    for i, c := range rest {
        if c == '/' {
            schema := rest[:i]
            table := rest[i+1:]
            if schema == "" || table == "" {
                return "", "", mcp.ResourceNotFoundError(uri)
            }
```

```
                return schema, table, nil
            }
        }
        return "", "", mcp.ResourceNotFoundError(uri)
    }
```

What works well:

- Both parsers (`parseDBResourceURI` , `parseEnumResourceURI`) validate that neither component is empty.
- Only the first `/` separator is used — additional slashes in a malicious URI would be included in the `table` component, causing a REST API 404 rather than unexpected behavior.
- Short-circuits on prefix length check.

Residual risk (low):

- Characters like `../` , `%2F` , null bytes are not filtered. They are forwarded to the Ichor REST API which will either reject them or route them to 404. No known exploitation path in the current stack, but this is undefended surface that relies on Go's HTTP router for safety.

---

4. Transport Security

4.1 MCP Transport (stdio)

Rating: Strong

Evidence (`main.go:69`):

```
if err := server.Run(ctx, &mcp.StdioTransport{}); err != nil {
```

What works well:

- stdio transport binds to no network port. It communicates exclusively via process stdin/stdout.
- Only the parent process (Claude Desktop, an agent runner) can communicate with the server. No remote connections are possible.
- No TLS configuration needed — the transport is inherently local and process-isolated.
- SSRF via MCP transport is not possible; an attacker would need local process access.

What is absent:

- **No authentication of the stdio caller**: Any process that can write to the server's stdin can invoke tools. This is acceptable for the intended deployment model (Claude Desktop as trusted parent), but would be a risk in multi-process server scenarios.

4.2 MCP → Ichor REST API Transport

Rating: Weak (development) / Conditionally Adequate (production)

Evidence (`main.go:27` , `client/ichor.go:26–29`):

```
apiURL := flag.String("api-url", "http://localhost:8080", ...)
```

```
httpClient: &http.Client{
    Timeout: 30 * time.Second,
```

```
    },
```

What works well:

- 30-second request timeout prevents indefinite blocking on an unresponsive backend.
- Request context propagation (`http.NewRequestWithContext`) allows in-flight cancellation if the agent or OS cancels the MCP session.
- Explicit `Accept: application/json` and `Content-Type: application/json` headers.

What is weak:

- **HTTP by default**: The `--api-url` default is `http://localhost:8080`. Nothing in the codebase enforces HTTPS for production deployments. Bearer tokens are transmitted in plaintext over HTTP.
- **No TLS certificate validation configuration**: There is no way to configure custom CA certificates or client certificates via the CLI. A deployment using self-signed certificates would need to accept all certificates (a security risk) or require changes to the transport.
- **No HTTPS enforcement**: An operator could misconfigure `--api-url http://production.ichor.company.com` and not realize tokens are transmitted unencrypted.
- **No request ID**: Each HTTP request to Ichor carries no correlation header (no `X-Request-ID`, no `X-Tool-Name`). Individual requests are indistinguishable in Ichor's access logs.

---

5. Reliability Protections

5.1 Error Propagation

Rating: Adequate (with information leakage concern)

Evidence (`client/ichor.go:58-62`):

```go
if resp.StatusCode < 200 || resp.StatusCode >= 300 {
    return nil, fmt.Errorf("HTTP %d: %s", resp.StatusCode, string(respBody))
}
```

What works well:

- HTTP errors (4xx, 5xx) are converted to Go errors and propagated as `errorResult("...")` to the agent.
- Agents receive actionable error messages that allow them to retry or adjust their approach.
- The 30-second timeout prevents resource exhaustion from hung connections.

Information leakage concern:

- The full HTTP response body from Ichor is included verbatim in the error message shown to the agent. If Ichor returns internal stack traces, SQL error messages, or debug information in its error body (which should not happen in production but might in development), that information would reach the agent's context. An agent under prompt injection could use this to exfiltrate diagnostic information.

5.2 Graph Traversal Safety

Rating: Adequate (cycles handled, depth unbounded)

Evidence (`workflow_graph.go:204-252`):

```go
if visited[nodeID] {
    *lines = append(*lines, prefix+"  ↻ (continues above)")
    return
```

```
    }
    visited[nodeID] = true
```

What works well:

- Both `generateFlowOutline` and `walkPathNode` maintain a `visited` map and detect cycles. A cycle causes the traversal to print `"↻ (continues above)"` and return, preventing infinite recursion.
- `calculateDepth` (BFS for `explain_workflow_node`) similarly marks visited nodes to prevent infinite loops.

Residual weakness:

- **No depth limit**: There is no maximum node depth or maximum traversal size enforced. A legitimate workflow with hundreds of nodes would traverse all of them. An adversarial workflow with pathological fan-out could generate an extremely large output and flood the agent's context window. The backend's workflow validation should reject non-DAG graphs, so this is a low-probability risk in practice.

---

6. Audit and Observability

6.1 MCP-Level Tool Call Logging

Rating: Absent

Evidence: There are no logging statements in `main.go`, any tool handler, resource handler, or prompt handler. No logger is initialized. No structured log output is produced.

Impact:

- An enterprise administrator has zero visibility into which tools were called, with what arguments, at what time, or by which agent session.
- If an agent creates a harmful workflow, there is no MCP-level audit trail to establish causality.
- Incident response is blind: "which tool call created this workflow?" cannot be answered from MCP logs because there are none.

6.2 Correlation to Ichor API Logs

Rating: Absent

Evidence (`client/ichor.go:33–63`): The `doRequest` method adds no correlation headers beyond `Authorization`, `Accept`, and `Content-Type`. There is no `X-Request-ID`, `X-Tool-Name`, `X-Agent-Session`, or `X-MCP-Context` header.

Impact:

- Ichor's access logs show authenticated REST calls but cannot distinguish MCP-originated requests from frontend or direct API calls.
- Cannot determine from Ichor logs: "this workflow was created by the MCP server" vs. "this was created via the frontend."
- Cannot reconstruct the sequence of tool calls that led to a configuration change.

6.3 Request-Level Correlation

Rating: Absent

No request ID is generated per tool call. In implementations where an agent makes multiple parallel tool calls (which some MCP clients support), the resulting HTTP requests are interleaved and unidentifiable.

Summary Assessment Matrix

Control Area	Control	Rating	Evidence Location
Authentication	Hard-fail startup on missing token	**Strong**	`main.go:32–38`
Authentication	Bearer header injection on every request	**Strong**	`client/ichor.go:41`
Authentication	CLI flag leaks token to process list	**Weak**	`main.go:29` — `--token` flag
Authentication	Claude Desktop config stores token in plaintext	**Weak**	`README.md:44–55`
Authentication	Token lifecycle management (expiry, rotation, revocation)	**Absent**	No refresh logic anywhere
Authorization	RBAC enforced by REST API, not bypassable by MCP	**Strong**	`introspectionapi/routes.go:34–65`
Authorization	Schema introspection gated to admin only	**Adequate**	`introspectionapi/routes.go:35–48`
Authorization	No per-tool scope at MCP server level	**Absent**	`register.go` — all tools offered regardless of role
Authorization	`search_enums` asymmetric auth (list=admin, values=any)	**Weak**	`search.go:78 vs 85`, `routes.go:54 vs 64`
Input Validation	Required field guards on all parameterized tools	**Adequate**	Every tool handler
Input Validation	UUID format validation	**Absent**	String spliced directly into URLs
Input Validation	Workflow validate-first by default	**Adequate**	`write_workflow.go:56–77`
Input Validation	`validate=false` bypass fully exposed	**Weak**	`write_workflow.go:43, 92`
Input Validation	No validate-first for UI writes	**Absent**	`write_ui.go` — no dry-run pattern
Input Validation	Resource URI parsing (empty check)	**Adequate**	`resources.go:150–187`
Input Validation	Resource URI path traversal character filtering	**Absent**	`resources.go` — no sanitization
Transport	stdio transport (no network exposure)	**Strong**	`main.go:69` — `StdioTransport`
Transport	30-second HTTP timeout	**Adequate**	`client/ichor.go:27`
Transport	HTTP default (no TLS enforcement)	**Weak**	`main.go:27` — `http://localhost:8080` default
Transport	No HTTPS configuration option	**Absent**	`client/ichor.go:22–30` — no TLS config
Reliability	Cycle detection in graph traversal	**Adequate**	`workflow_graph.go:231–234`
Reliability	Unbounded graph traversal depth	**Weak**	`workflow_graph.go` — no depth limit
Reliability	Error propagation to agent (with full body)	**Adequate**	`client/ichor.go:58–62`
Audit	MCP tool call logging	**Absent**	No logger in `main.go` or any handler
Audit	Correlation headers to Ichor API	**Absent**	`client/ichor.go:40–45` — no `X-Request-ID`

Control Area	Control	Rating	Evidence Location
Audit	Agent-session attribution in logs	**Absent**	No session ID concept anywhere in MCP stack

Key Findings Summary

Strongest protections:

1. Hard-fail on missing token at startup prevents any accidental unauthenticated deployment
2. stdio transport eliminates network attack surface entirely — no open ports to probe
3. `auth.RuleAdminOnly` on schema introspection ensures the most sensitive introspection requires elevated credentials
4. The validate-first default on `create_workflow` and `update_workflow` catches structural errors before writes commit

Most significant weaknesses:

1. **Complete absence of audit logging** at the MCP layer — the most impactful gap for enterprise deployments. Every tool call is invisible to administrators.
2. **No token lifecycle management** — session failures on expiry, no rotation support
3. **`validate=false` exposed as a user-facing parameter** — developer escape hatch should be admin-gated or removed
4. **No correlation headers** — MCP-sourced changes cannot be distinguished from other API clients in Ichor's logs
5. **HTTP default (no TLS enforcement)** — tokens transit in plaintext if `--api-url` points to a non-localhost host